

LECTURE - 22

Tuesday
4/5/26

□ Contiguous Memory Allocation:

⇒ Multi-programming w fixed Task: Each segment has
a process loaded in it and each segment will have
a queue of fixed size each.

- It will depend on size key ham 80KB wala process
100KB wala segment main jana chia instead of 300KB
segment so low "internal Fragmentation"
- Problem is that if 300KB wala segment is free and 80KB
wala isn't going into that segment because it's too big for it;
then despite having an empty ~~there~~ segment process is waiting
- Solution is to make only one queue (single) for all segments.

⇒ Multiple Programming w variable Tasks: Jobs can move here and there with no boundary ∴ No Internal Fragmentation.

- It will cause external Fragmentation if a memory hole created due to a bach wala process being terminated.

- External Frag: Space is present for a process but free space isn't contiguous & choti choti space banni hui which are holes.

Solution ⇒ Memory Compaction: Swap Holes with data above it so a contiguous space ^{needed} ↑

- We can fit processes in hole if they can, wrna phir memory compaction krni gay bcs it is expensive process.

⇒ Paging:

- we don't need contiguous memory here. Processes could be divided into small ^{equal} chunks which are "Pages" and RAM ~~must~~ must be divided into same equal chunks called "Frames".

- Therefore any page (chunk of process) could be loaded in any Frame (chunk of RAM) ^{particular}

- Therefore only (No. of Frames should be available in RAM equivalent to No. of Pages (of process) so each ^{page} ~~frames~~ maps to frame anywhere in the memory (not contiguous)

- Page Table is used to keep track of wherever the page is mapped to frame. Each process w separate table. (control Bits here)

- Size of table is in 2's power

- Every logical address is divided with two parts:
page no. (P) & offset (d) → which part/instruction of page. ^{specific}

$\boxed{P \mid d}$

Out of 20 bits; more will be given to page no.

- To make physical address; we'll find frame (f) of that page no. from page table ∴ $\boxed{f \mid d}$
And Offset (d) remains same.

- Pages could be shared to child or other processes to reduce cost of Fork().

Continue ... Lecture: 22

⇒ Virtual Memory: It uses "demand page" to be implemented.

- We can borrow storage from secondary storage for RAM but that 2ndary storage is never used as RAM.
- ^{Heavy} Applications ~~can~~ could be loaded in small RAM computer as a part of 2ndary is given to it; tho it will be slower as moving back & forth b/w RAM & 2ndary storage is time consuming (as 2ndary storages are very slow).

→ Demand Paging: Same as paging but we've swap in and swap out operations. Only extra part of process/app. which isn't loaded in RAM but is needed; is sent to 2ndary memory & swapped when no longer ~~needed~~ needed.

• 'Pager' swaps the pages.

- That is why we've Valid column in the page table which have value = 1 ~~if~~ if page is in the memory ^(RAM); else value = 0 means page is not in the memory (RAM) and is in 2ndary storage.
- -1 means legal too (means that page belongs to this process); 0 is illegal.
- In 2ndary storage; all parts of processes remains there and only they get copied to RAM.

That is why we have dirty bit column which has bit '0' means nothing change in that part of process ∴ we don't need to send it back to 2ndary storage (when not needed) as uska purana version is already in 2ndary memory. If dirty bit = 1; then we'll over write 2ndary memory with RAM version as stuff has changed.

→ Page Fault: If the valid bit of a page is zero; means that page is invalid ∴ page fault occurs.

Therefore we need to find a free space in RAM and bring that invalid instruction there. Now page table's valid bit gets 1 and frame column has frame no. where new instr. loaded. That instruction is restarted now.

- If we don't get free space in RAM for page fault; we'll find a victim page & remove it from memory. To find the page to be replaced; we'll use an algorithm:

⇒ FIFO Page Replacement: Remove the oldest page as victim

Input: 7 0 1 2 0 3 0 4 2 3 0 2 1

	7	7	7	2		2	2	4	4	4	0		0
		0	0	0		3	3	3	2	2	2		1
			1	1		1	0	0	0	3	3		3

Annotations: "skipped as 0 already in queue" points to the 0 at index 5; "skipped" points to the 0 at index 11.

Method:

- we replaced the new input i.e.: 2 with '7' as 7 was in the queue for longest (3 instance of it and 2 instance of 0 and 1 instance of 1)
- If input is already in queue we'll skip (like: 2).